PRACTICAL - 3

Aim

Implement A* or heuristic search for 8-puzzle/path planning and evaluate the effect of different heuristics.

Theory

A* is a heuristic search algorithm that selects the most promising state using:

$f(n)=g(n)+h(n)$

Where:

- **g(n)** = cost from the initial state to the current state.
- **h(n)** = estimated cost from the current state to the goal.
- **f(n)** = total estimated cost.

Two commonly used heuristics for the 8-puzzle are:

1. Misplaced Tiles

Counts the number of tiles that are not in their correct position.

h(n)=Number of misplaced tiles

2. Manhattan Distance

Calculates the total distance of each tile from its goal position.

$h(n)=\sum|x1-x2|+|y1-y2|$

Comparison: Manhattan Distance generally provides better guidance than Misplaced Tiles because it considers how far each tile is from its correct position. Both heuristics are admissible for the standard 8-puzzle, so A* can still find an optimal solution.

Program

```
import heapq
```

```
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

# Misplaced Tiles heuristic
def misplaced_tiles(state):
    return sum(state[i] != goal[i] and state[i] != 0
               for i in range(9))


# Manhattan Distance heuristic
def manhattan_distance(state):
    distance = 0

    for i, tile in enumerate(state):
        if tile != 0:
            current_row, current_col = divmod(i, 3)
            goal_index = goal.index(tile)
            goal_row, goal_col = divmod(goal_index, 3)

            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)
```

```
    return distance


# Generate possible moves
def get_neighbors(state):
    neighbors = []

    zero = state.index(0)
    row, col = divmod(zero, 3)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_zero = new_row * 3 + new_col

            new_state = list(state)
            new_state[zero], new_state[new_zero] = \
                new_state[new_zero], new_state[zero]
```

```
            neighbors.append(tuple(new_state))

    return neighbors


# A* Search
def a_star(start, heuristic):
    priority_queue = []
    heapq.heappush(priority_queue, (heuristic(start), 0, start, [start]))

    visited = set()

    while priority_queue:

        f, g, state, path = heapq.heappop(priority_queue)

        if state == goal:
            return path

        if state in visited:
            continue
```

```
        visited.add(state)

        for next_state in get_neighbors(state):
            if next_state not in visited:

                new_g = g + 1
                new_f = new_g + heuristic(next_state)

                heapq.heappush(
                    priority_queue,
                    (new_f, new_g, next_state, path + [next_state])
                )

    return None


# Initial puzzle
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

# A* using Misplaced Tiles
path1 = a_star(start, misplaced_tiles)
```

```
# A* using Manhattan Distance
path2 = a_star(start, manhattan_distance)

print("Misplaced Tiles Heuristic")
print("Number of moves:", len(path1) - 1)

print("\nManhattan Distance Heuristic")
print("Number of moves:", len(path2) - 1)
```

output :

```
A* Solution:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Heuristic Evaluation

In A* search, different heuristics can affect the efficiency of finding the solution. The Misplaced Tiles heuristic counts the number of tiles that are not in their correct positions. The

Manhattan Distance heuristic calculates the total distance of each tile from its correct position. Manhattan Distance usually performs better because it provides more information about how far the current state is from the goal. Both heuristics can find an optimal solution when they do not overestimate the actual cost.

Conclusion

Thus, the A* search algorithm was successfully implemented to solve the 8-puzzle problem. The algorithm uses the cost function f(n) = g(n) + h(n) and the Manhattan Distance heuristic to efficiently find an optimal solution path.